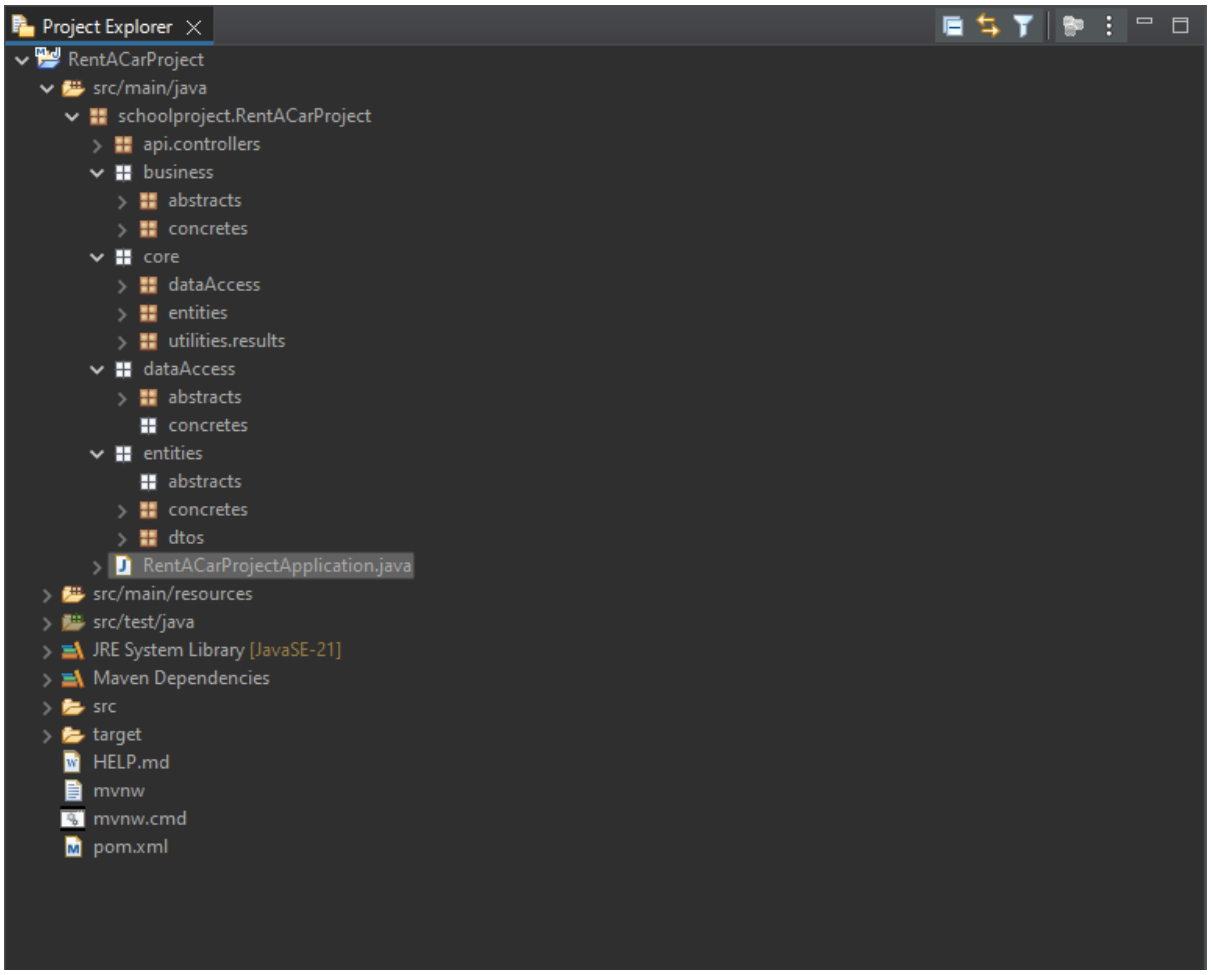
NESNE DAYALI PROGRAMLAMA DERSİ PROJE ÖDEVİ

Geliştirdiğimiz otomobil kiralama sitesinin aşamalarını ekran görüntüleriyle göstereceğiz.



Projemizde böyle katmanlı bir mimari kullandık. Katmanları sırayla açıklayalım:

-Entity : Varlıkların oluşturulduğu katmandır. Projeye ait gerekli, veritabanında tablosu oluşturulacak tüm varlıklar burada tanımlanır.

-Data Access : Veritabanı işlemlerinin yapıldığı katmandır. Oluşturduğumuz tabloların özelliklerini hangi şartlarda getireceğimiz ve API'ye nasıl ileteceğimiz burada belirtilir.

-Core : Evrensel kodlamaların yapıldığı bir katmandır. Bu katmanda yazılan kodlar projeye özgü olmaz ve başka projelerde de kullanılabilir.

-Business : İş kodlarının yazıldığı katmandır. Varlıkları ekleme, çıkarma, güncelleme ve bir sürü diğer işlemleri hangi kurallara göre yapacağımızı belirleriz.

-API : Burada yazdığımız kodların dış dünya ile ilişkisinde kapı görevi gören bir arayüzdür.

```
@Data
@Entity
@Table(name = "cars") //Hangi tabloya karşılık geldiğini yazıyoruz.
@AllArgsConstructor //Parametreli constructor için
@NoArgsConstructor //Parametresiz constructor için
public class Car {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY) //Birer birer arttığını
gösterir.
@Column(name = "id")
private int id;

//@Column(name = "brand_id")
//private int brandId;

//@Column(name = "model_id")
//private int modelId;

//@Column(name = "color_id")
//private int colorId;

@Column(name = "description")
private String description;

@Column(name = "model_year")
private int modelYear;

@Column(name = "daily_price")
private double dailyPrice;

@ManyToOne()
@JoinColumn(name = "brand_id")
private Brand brand;

@ManyToOne()
@JoinColumn(name = "model_id")
private Model model;

@ManyToOne()
@JoinColumn(name = "color_id")
private Color color;

}
```

Bu Entity katmanından “Car” sınıfına örnektir. Spring Boot kurmamız sayesinde çoğu veritabanı işlemini kodla yazmak yerine bir etiket ile halledebildik. Öncelikle Araba’ya ait gerekli özellikleri tanımladık. En üstte Lombok kurmamız sayesinde gelen bir özellikle @Data etiketi tanımladık ve bu da getter-setter işlemlerini direkt yazmamıza gerek olmadan yapmayı sağladı. @AllArgsConstructor ve @NoArgsConstructor etiketleriyle parametreli ve parametresiz constructorları tanımladık. @Table etiketi ile bu sınıfın veritabanındaki “cars” tablosuna karşılık geldiğini belirttik. Diğer özelliklerin üzerindeki @Column etiketi de o tablonun satırlarına karşılık geliyor. brand_id, model_id ve color_id bu araba tablosunda foreign_key görevi görmektedir. Çünkü bu üçü de ayrı bir tabloya sahiptir. Bu id’leri düz bir şekilde tanımlamayıp sınıflarından nesne oluşturarak getirdik ve isim olarak id’lerine eşitledik. @JoinColumn ile birden fazla tabloyu birleştirmiş olduk.

```
public interface CarDao extends JpaRepository< Car , Integer> {
 Car getByDescription(String description);
 Car getById(int id);
 Car getByModelYear(int modelYear);
 Car getByDailyPrice(double dailyPrice);
 @Query("SELECT c FROM Car c WHERE c.id = :id AND c.brand.id = :brandId")
 List<Car> getByIdAndBrand(@Param("id") int id, @Param("brandId") int
brandId);
 @Query("SELECT c FROM Car c WHERE c.id = :id AND c.model.id = :modelId")
 List<Car> getByIdAndModel(@Param("id") int id, @Param("modelId") int
modelId);
 @Query("SELECT c FROM Car c WHERE c.id = :id AND c.color.id = :colorId")
 List<Car> getByIdAndColor(@Param("id") int id, @Param("colorId") int
colorId);
 List<Car> getByModelIn(List<Integer> models);
 List<Car> getByColorIn(List<Integer> colors);
 List<Car> getByBrandIn(List<Integer> brands);
 List<Car> getByDescriptionContains(String description);
 List<Car> getByDescriptionStartsWith(String description);
 @Query("SELECT new
schoolproject.RentACarProject.entities.dtos.CarWithBrandDto(c.id, b.id,
c.description, b.name, c.dailyPrice) FROM Brand b INNER JOIN b.cars c")
 List<CarWithBrandDto> getCarWithBrandDetails();
 @Query("SELECT new
schoolproject.RentACarProject.entities.dtos.CarWithColorDto(c.id, co.id,
c.description, co.name, c.dailyPrice) FROM Color co INNER JOIN co.cars c")
 List<CarWithColorDto> getCarWithColorDetails();}
```

CarDao sınıfımız JpaRepository'den implement edilmiştir. Bu Spring Boot tarafından gelmektedir. Bu sayede tüm ekleme, çıkarma gibi işlemleri arka alanda yapabiliyor. Biz burada veritabanı ile ilgili işlemleri uyguluyoruz. Bunları kodumuzu çalıştırdıktan sonra Swagger sitesinde deneyebiliyoruz. Verilerimize ekleme de yapabilmekteyiz. Birden fazla tabloyu birleştirip verileri döndürebilmek için de @Query etiketiyle özel SQL sorguları yazıyoruz. Sadece tek bir veri döndürmediklerimizi 'List' ile döndürüyoruz.

```
public class Result {
private boolean success;
private String message;

public Result (boolean success) { //Sadece true/false döndürmek
isteyebiliriz.
this.success= success;
}

public Result (boolean success, String message) {
this(success);
this.message = message; //ilgili nesneleri yapılandırmış olduk.
}
public boolean isSuccess() { //Dışarıdan erişim için uygulandı.
return this.success;
}
public String getMessage() {
return this.message;
}
}
```

Core katmanında Result sınıfı oluşturduk. Bu veri işlemlerinde olumlu olumsuz dönüş almamız için uygulanmıştır. Message ve success tanımladık. Hem message hem de success döndürmek istiyorsak veya message döndürmek istemiyorsak onlar için ayrı ayrı constructor oluşturduk. İki farklı metot oluşturarak message ve success nesnelerini return ediyoruz.

```
public class SuccessResult extends Result {
public SuccessResult() {
super(true);
}

public SuccessResult(String message) {
super(true,message);
}
}
```

Bir SuccessResult oluşturduk ve Result dan inherit edildi. Böylece oradaki tüm özellikleri buraya tanımladık. True/False dönüşünü kesinlikle true olmaya ayarladık ve mesaj veya mesajsız dönüşü için iki farklı constructor oluşturduk. “super” komutuyla base sınıfın(Result) constructor’larını çalıştırıyoruz. Aynı şekilde bir tane de ErrorResult oluşturup oraya da true/false dönüşünü false olarak ayarladık. Bu sayede Business katmanındaki ve Controller katmanındaki işlemlerde olumlu dönüşleri direkt bu sınıflarla tanımlayabiliyoruz. Ayrıca eğer bir List<Car> döndürülecekse bunlar için de DataResult tanımlandı.

```
public class DataResult<T> extends Result { //inheritance kullanıldı.

private T data;

public DataResult(T data, boolean success, String message) {
super(success, message); //super ile base sınıfın (Result)
constructorlarını çalıştırmaya yarıyor.
this.data = data; //set etmiş olduk.
}

public DataResult(T data, boolean success) { //message döndürmek
istemeyebiliriz.
super(success);
this.data = data;
}

public T getData() { //Data bilgisini okumak için get etmemiz de lazım
return this.data;
}
```

Generic Tipler kullanıldı ve <T> yazılmasının sebebi ne döndüreleceği net olmadığındandır. T, “type”dan gelmektedir fakat başka bir harf de yazılabilir.

Sonra Business katmanında CarService adında bir interface açtık. İçine CarDao’da yazdıklarımızı uyguladık.

```
DataResult<Car> getByDescription(String description);
DataResult<Car> getById(int id);
DataResult<Car> getByModelYear(int modelYear);
```

Örneğin bir özellikten tek bir değer döndüreceğimiz zaman böyle yazıyoruz. Core katmanında oluşturduğumuz Result sınıflarını da burada uyguluyoruz. Böylece API işlemlerini uygulurken olumlu/olumsuz cevaplar alabiliyoruz.

```
@Service
public class CarManager implements CarService {

private CarDao carDao;

@Autowired
public CarManager(CarDao carDao) {
super();
this.carDao = carDao;
}

@Override
public DataResult<List<Car>> getAll() {
return new SuccessDataResult<List<Car>>
(this.carDao.findAll(),("Data Listelendi."));
}

public Result add(Car car) {
this.carDao.save(car);
return new SuccessResult("Araba Eklendi.");
}

public DataResult<List<Car>> getByIdAndBrand(int id, int brandId) {
return new SuccessDataResult<List<Car>>
(carDao.getByIdAndBrand(id, brandId),"Data Listelendi");
}
```

CarManager sınıfından bir örnek buradadır. Öncelikle CarService implement ettik. Böylece oradaki imza metotları kullanabiliyoruz. Ayrıca CarDao'yu Dependency Injection kullanarak buraya ekliyoruz ve böylece katmanlar arası erişimi sağlıyoruz. Bu sayede oradaki metotlar burada uygulanabiliyor. @Service etiketiyle Spring Boot'a bu sınıf hakkında bilgilendirme yapıyoruz.

@Autowired etiketiyle de Spring arka planda buna karşılık gelebilecek, carDao nun instance i olabilecek bir class üretiyor.

Son olarak Controller sınıfı oluşturduk.

```
@RestController
@RequestMapping("api/cars")
@CrossOrigin
public class CarsController {
```

@RestController ile bu sınıfın Controller görevi üstlendiğini Spring'e ilettik. @RequestMapping etiketi sayesinde sınıfın eşleştiği URL'yi belirtiyoruz.

```
private CarService carService;
@Autowired
public CarsController(CarService carService) {
super();
this.carService = carService;
}
```

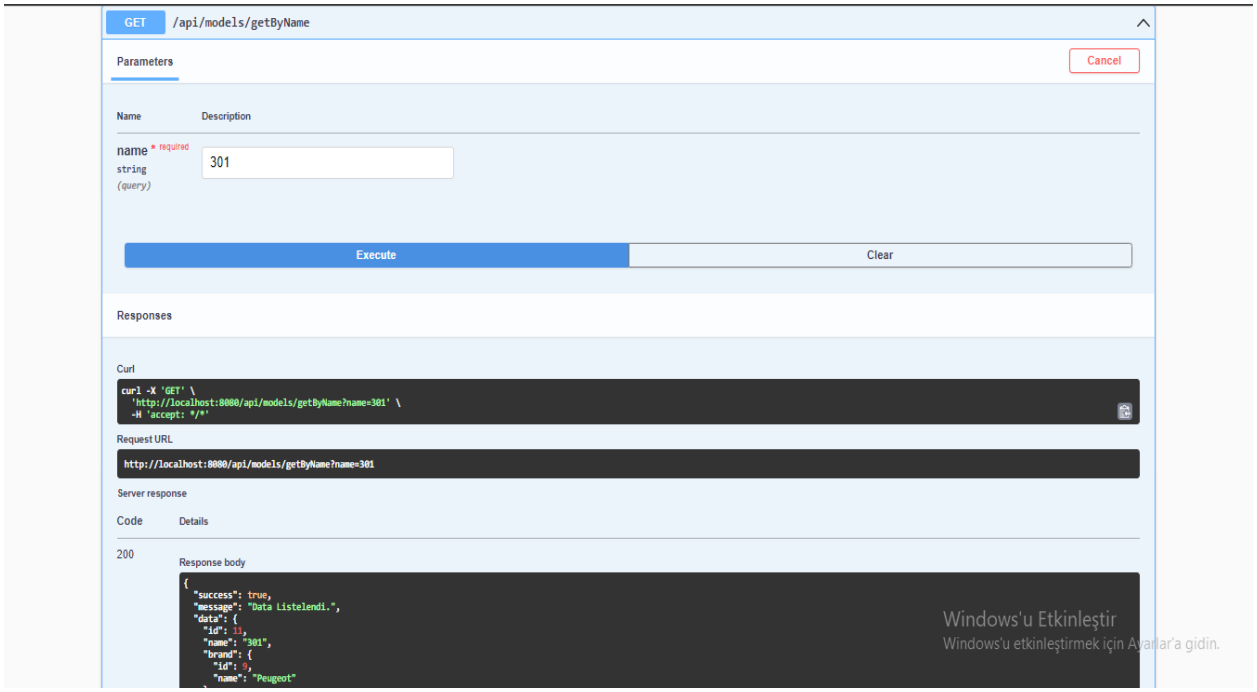
Burada da aynı şekilde CarService, Dependency Injection ile eklendi. Böylece CarManager'da yazılan kurallar buraya bağlanır ve bu kodlar da uygun yazıldığı dille API'ye iletilir. API'den de Frontend Programlama Dilimize iletilir ve bu sayede sitemizde tüm veriler sıralanır.

```
     @GetMapping("/getall") // api/cars/getall
public DataResult<List<Car>> getAll() {
return this.carService.getAll();
}
@PostMapping(value = "/add")
public ResponseEntity<?> add(@Valid @RequestBody Car car) {
return ResponseEntity.ok(this.carService.add(car));
}
@GetMapping("/getByDescriptionContains")
public DataResult<List<Car>> getByDescriptionContains(@RequestParam String
description){
return this.carService.getByDescriptionContains(description);
}
@GetMapping("/getByIdAndModel")
public DataResult<List<Car>>
getByIdAndModel(@RequestParam("id") int id,@RequestParam("modelId") int
modelId){
System.out.println(id);
System.out.println(modelId);
return this.carService.getByIdAndModel(id, modelId);
}
```

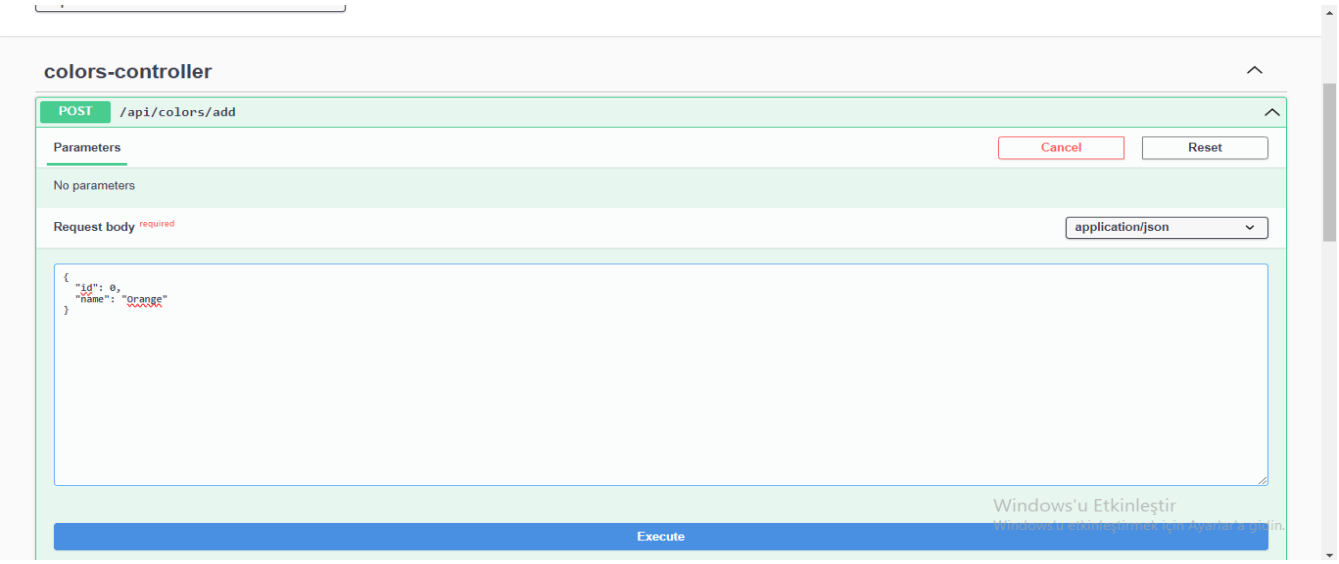
Sınıftan bir örnek ekledik. Burada @GetMapping ve @PostMapping ile metotların işlevini belirttik. Get ile verileri getiriyoruz ve Post ile ekleme yapabiliyoruz. Ayrıca yanlarında yazdığımız Route adları da metot adımızla aynı veya benzer olmalıdır.

@RequestParam ile HTTP isteğimizin parametrelerini alıyoruz. Bu anotasyon ile parametreleri kolayca alabiliyoruz.

@RequestBody, Spring Framework'ün MVC modülünde kullanılan bir anotasyondur. Bu anotasyon, HTTP isteğinin gövdesinden gelen veriyi almak için kullanılır.



Swagger uygulamasından bir örnek. api/models/getByName ile 301 modelini giriyoruz ve bize o modele ait tüm özellikleri gösteriyor.



Başka bir örnekte de api/colors/add ile renk eklemeyi gösteriyoruz. Orange rengini örnek olarak yazdık ve Execute dediğimizde bu renk veritabanımızdaki colors tablosuna eklendi. Biz de buna karşılık şu mesajı aldık :

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/colors/add' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 0,
  "name": "Orange"
}'
```

Request URL

```
http://localhost:8080/api/colors/add
```

Server response

Code Details

200

Response body

```
{
  "success": true,
  "message": "Renk eklendi."
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Thu,09 May 2024 15:14:50 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Windows'u Etkinleştir
Windows'u etkinleştirmek için Ayarlar'a gidin.

Responses

Code Description Links

Result yazmamız sayesinde eklediğimiz renkte bir true karşılığı aldık ve “Renk eklendi.” mesajı aldık. Sonrasında da renk, id = 9 olarak eklendi.


```
  },
  {
    "id": 4,
    "name": "Blue"
  },
  {
    "id": 5,
    "name": "Green"
  },
  {
    "id": 6,
    "name": "Yellow"
  },
  {
    "id": 7,
    "name": "Grey"
  },
  {
    "id": 8,
    "name": "Brown"
  },
  {
    "id": 9,
    "name": "Orange"
  }
]
}
```

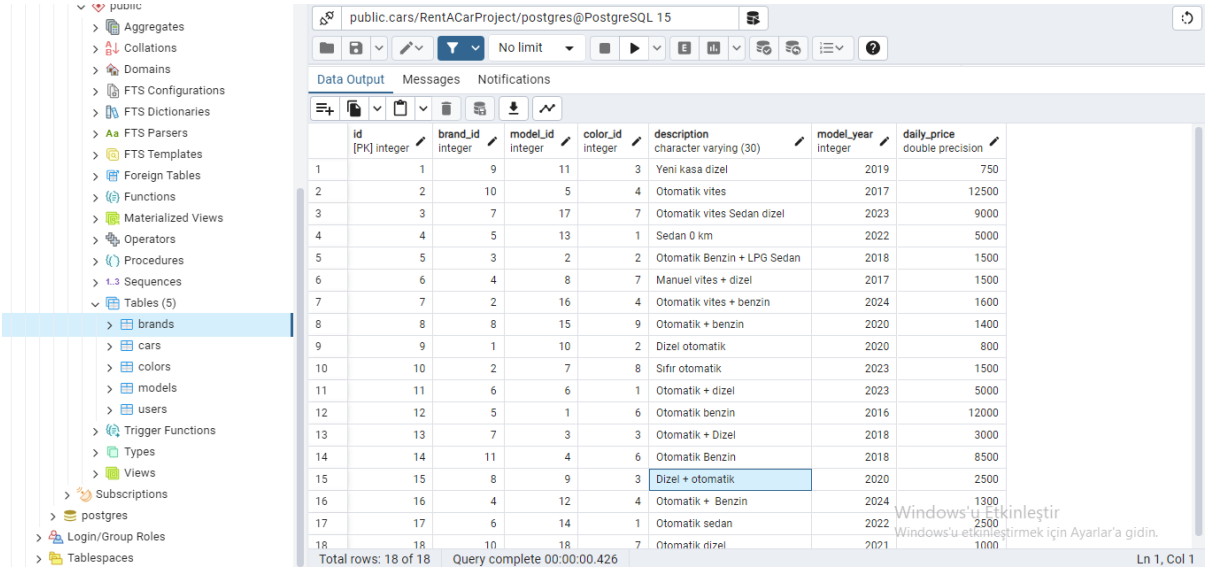

Download

Windows'u Etkinleştir
Windows'u etkinleştirmek için Ayarlar'a gidin.

Response headers

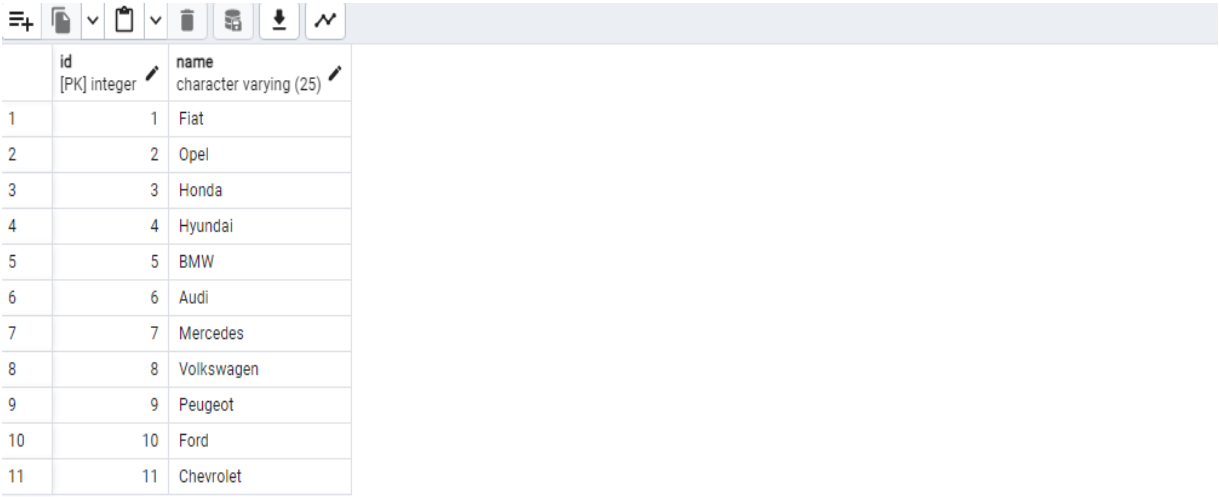
```
connection: keep-alive
content-type: application/json
date: Thu,09 May 2024 15:15:04 GMT
```

Bunun haricinde iki farklı tablonun id’lerini çağırmayla tek bir veriyi getirme, mail’e göre kullanıcı getirme gibi daha bir sürü işlemi burada uygulayabiliyoruz.



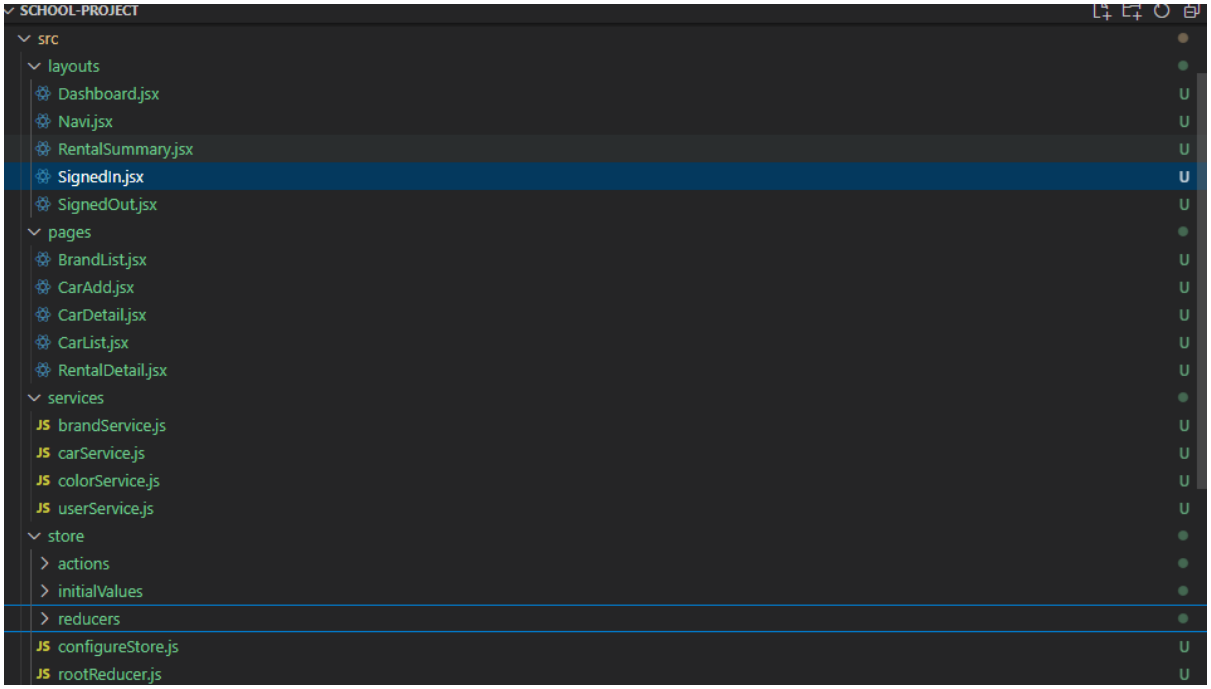
	id [PK] integer	brand_id integer	model_id integer	color_id integer	description character varying (30)	model_year integer	daily_price double precision
1	1	9	11	3	Yeni kasa dizel	2019	750
2	2	10	5	4	Otomatik vites	2017	12500
3	3	7	17	7	Otomatik vites Sedan dizel	2023	9000
4	4	5	13	1	Sedan 0 km	2022	5000
5	5	3	2	2	Otomatik Benzin + LPG Sedan	2018	1500
6	6	4	8	7	Manuel vites + dizel	2017	1500
7	7	2	16	4	Otomatik vites + benzin	2024	1600
8	8	8	15	9	Otomatik + benzin	2020	1400
9	9	1	10	2	Dizel otomatik	2020	800
10	10	2	7	8	Sıfır otomatik	2023	1500
11	11	6	6	1	Otomatik + dizel	2023	5000
12	12	5	1	6	Otomatik benzin	2016	12000
13	13	7	3	3	Otomatik + Dizel	2018	3000
14	14	11	4	6	Otomatik Benzin	2018	8500
15	15	8	9	3	Dizel + otomatik	2020	2500
16	16	4	12	4	Otomatik + Benzin	2024	1300
17	17	6	14	1	Otomatik sedan	2022	2500
18	18	10	18	7	Otomatik dizel	2021	1000

Veritabanı olarak da PostgreSQL kullanıldı. Tabloların verileri burada girildi ve marka, renk ve modeller oluşturuldu. En sonunda da bu id'ler doğru şekilde girilerek arabaları oluşturduk. Bunun yanında açıklama, model yılı ve günlük fiyatlar da girildi.



	id [PK] integer	name character varying (25)
1	1	Fiat
2	2	Opel
3	3	Honda
4	4	Hyundai
5	5	BMW
6	6	Audi
7	7	Mercedes
8	8	Volkswagen
9	9	Peugeot
10	10	Ford
11	11	Chevrolet

Bu da marka tablosunun oluşturulan verilerinin örneğidir.



Kullanıcı arayüzünü oluşturmak için React kütüphanesini kullandık. Burada Layouts, Pages, Services gibi dosyalar oluşturduk. Services dosyalarımızda fonksiyonlar oluşturarak API'den gelen URL'leri kullanarak verileri çağırmayı sağladık. Layouts da ise soldaki marka listelenmesi ile yukarıdaki Navbar ayarlarını yaptık.

BrandList ve CarList dosyalarında onlara ait Service'ları çağırdık.

```
const [brands, setBrands] = useState([]);
  useEffect(() => {
    let brandService = new BrandService();
    brandService.getBrands().then(result => setBrands(result.data.data))
  }, []);
```

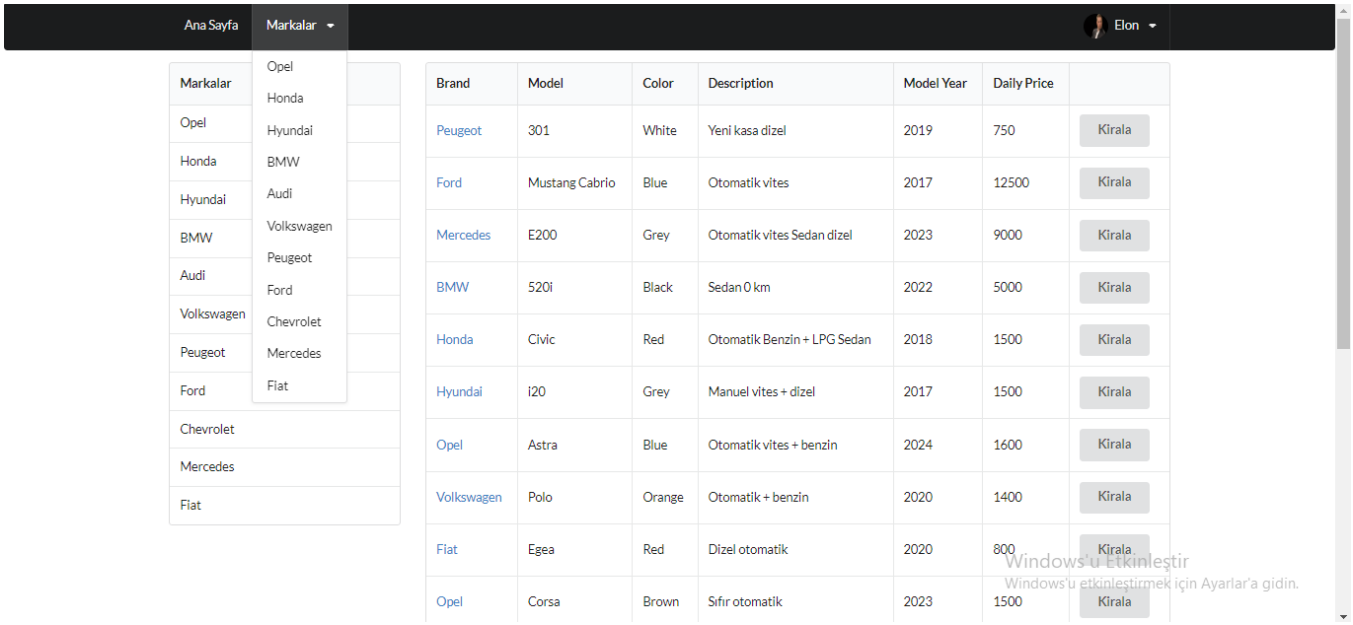
Örneğin BrandList üzerinden gidelim. O BrandService'deki tüm markaları getiren fonksiyonu(getBrands) çağırdık ve useState sayesinde oluşturduğumuz brands nesnesini useEffect ile fonksiyonumuza atadık ve data'ları da döndürdük.

```
<Table celled>
        <Table.Header>
          <Table.Row>
            <Table.HeaderCell>Markalar</Table.HeaderCell>
          </Table.Row>
        </Table.Header>
        <Table.Body>
          {brands.map((brand) => (
            <Table.Row>
              <Table.Cell>{brand.name} </Table.Cell>
            </Table.Row>
          ))}
        </Table.Body>
      </Table>
```

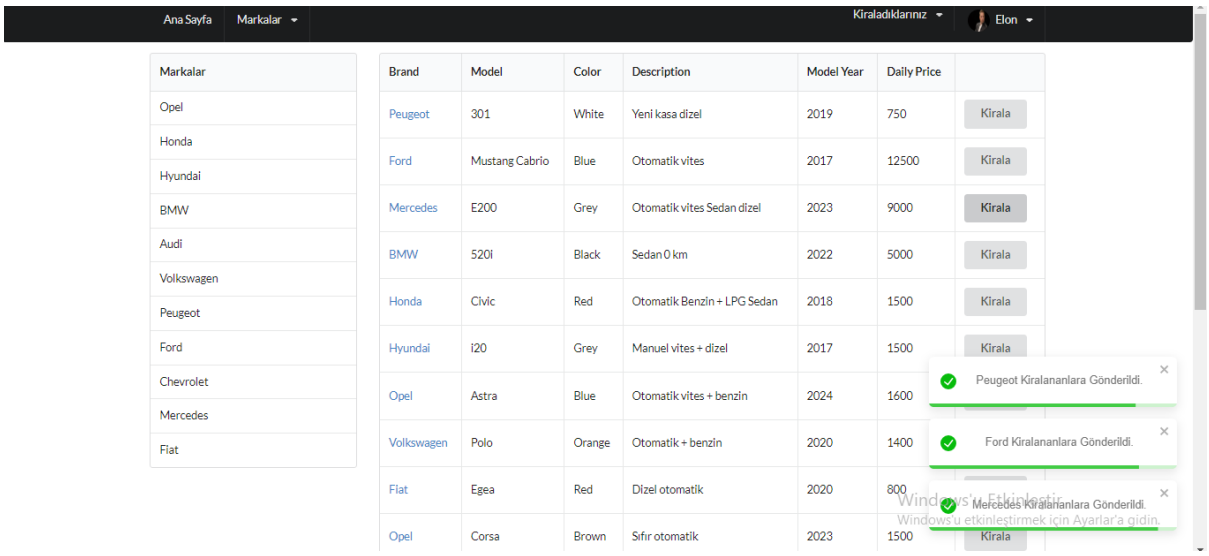
Burada map metodu sayesinde oluşturduğumuz “brand” içinde brand.name yazarak veritabanındaki markaları id sıralamasıyla aşağıya doğru tablo içinde yazdırıyoruz.

```
          <GridColumn width={4}>
            <BrandList />
          </GridColumn>
          <GridColumn width={12}>
            <Routes>
              <Route exact path="/" element={<CarList />} />
              <Route exact path="/cars" element={<CarList />} />
              <Route exact path="/cars/:brandName" element={<CarDetail />} />
              /<Route exact path="/brands" element={<BrandList />} />
              <Route path="/rental" element={<RentalDetail />} />
              <Route path="/car/add" element={<CarAdd/>} />
            </Routes>
          </GridColumn>
```

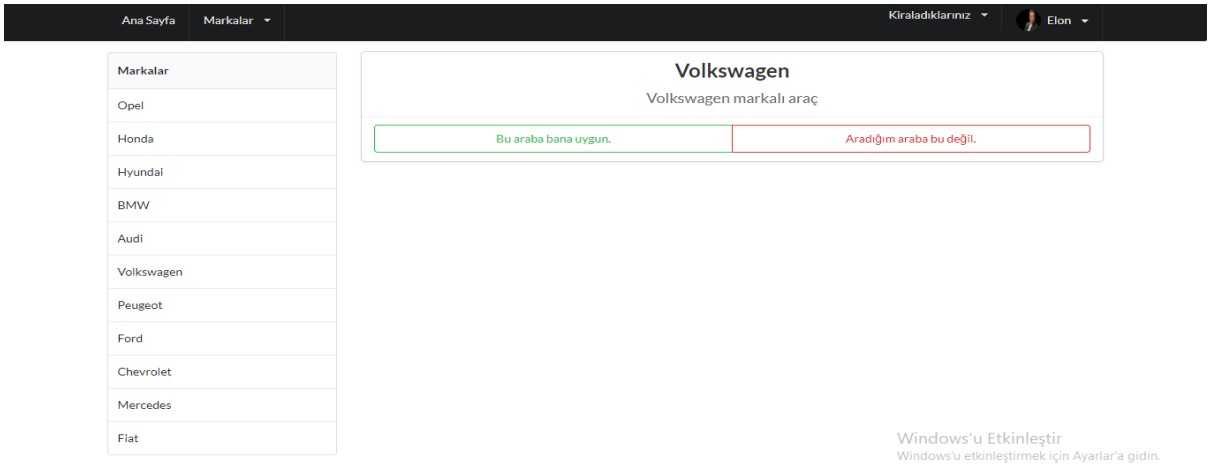
Dashboard dosyamızda da bağlantıları ayarlıyoruz. Basacağımız tuşların bizi hangi kısma götüreceğini aslında burada belirliyoruz. Ayrıca siteye bu path’leri eklediğimizde de bizi o kısımlara götürebilmesini sağlıyoruz.



Sitemiz şu şekildedir. Soldaki ve üstteki markalar ile genel olarak tüm özellikleriyle birlikte arabalarımız tamamen veritabanında oluştuğu şekliyle sıralanmıştır.



Kirala tuşuna basıldığında sağ altta çıkan mesajla birlikte sağ üstte Kiraladıklarınız kısmı çıkmaktadır. Oradan Kiraladıklarına Git diyerek ilerleyebiliyoruz.



Ana sayfadaki genel tablomuzda markalardan birine tıkladığımızda böyle bir bilgilendirme sayfası açılıyor. Bu da CarDetail kısmına gönderiyor bizi. Aynı şekilde her markanın isminin yazması da veritabanı ve Java kodları arasındaki ilişkiden gelmektedir.

Kısacası biz bu projede Java programlama dilinde yazdığımız kodlar ile veritabanı ve dış dünya arasında ciddi bir ilişki kurulmasını sağladık. Verileri belirli kurallara koyduk ve kendi içlerinde de iletişim halinde tuttuk. Sonra bu verileri API yardımıyla Frontend kısmına ilettik ve React aracılığıyla bunu bir websitesine dönüştürdük.